INFORME DE FIN DE UNIDAD

Datos Generales

Asignatura	Base de datos
Ciclo	3 A
Unidad	1
Resultado de aprendizaje de la unidad	Modela bases de datos de problemas reales, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Nombre del Docente	Edwin René Guamán Quinche
Fecha	04/11/2026
Enlace de Git	bdd-inventario_raintamia_plus
Nombres y apellidos del estudiantes	Isaac David Ramón Tinoco

1. TITULO

Base de datos para el registro de inventario de la embotelladora de agua RainTamia Plus.

2. CONTEXTO DEL PROBLEMA

Situación o entorno. RainTamia Plus es una PYME embotelladora de agua que produce y distribuye botellas, botellones y pacas de distintas presentaciones (por ejemplo, paca de 20 unidades de 600 ml). Su operación depende de materia prima como envases, tapas, sellos, etiquetas y fundas, y de productos terminados listos para la venta.

Dificultades sin una base de datos. La gestión actual del inventario se hace con métodos manuales, lo que genera tres problemas principales: (a) no hay control de reabastecimiento, pues no existe un mecanismo que detecte cuándo el stock cae por debajo del mínimo y solicite la compra; (b) hay opacidad en la trazabilidad de pérdidas, porque no se conserva un historial formal de movimientos como "daño" o "pérdida", sobre todo en la distribución; y (c) el valor monetario del inventario no está actualizado, lo que afecta la información que usa el área financiera. Como consecuencia, la empresa puede no cumplir pedidos por falta de insumos o tener insumos almacenados que no se usan para producir.

A quiénes afecta. Al encargado de inventario, que registra los movimientos; al vendedor, que necesita consultar stock disponible; al contador, que requiere costos y valoración confiables; y al propietario, que autoriza las compras y toma decisiones con base en las existencias reales.

Datos a gestionar. Los ítems de inventario (materia prima y productos terminados) con su stock actual, stock mínimo y costo o precio, las bodegas donde se almacenan, los encargados que registran los cambios y los movimientos (entrada, salida, ajuste y pérdida) con fecha, responsable y motivo. Gestionarlos correctamente es clave para la transparencia y la responsabilidad: permite saber quién registró cada movimiento, cuándo y por qué, y respaldar las decisiones de reabastecimiento y las pérdidas con evidencia.

3. OBJETIVOS

3.1. Objetivo General

Desarrollar una base de datos relacional en PostgreSQL para el registro del inventario de materia prima y productos terminados de la embotelladora RainTamia Plus, con fines de control de existencias y trazabilidad de movimientos.

3.2. Objetivos Específicos

- Modelar el inventario de RainTamia Plus mediante el diagrama Entidad-Relación y el modelo relacional, con sus entidades, atributos, relaciones y cardinalidades.
- Implementar el esquema físico en PostgreSQL, con sus datos de ejemplo y cinco consultas de álgebra relacional con su equivalente en SQL.

4. MODELOS CONCEPTUALES DE BASES DE DATOS

4.1. Modelo Entidad-Relación (E-R)

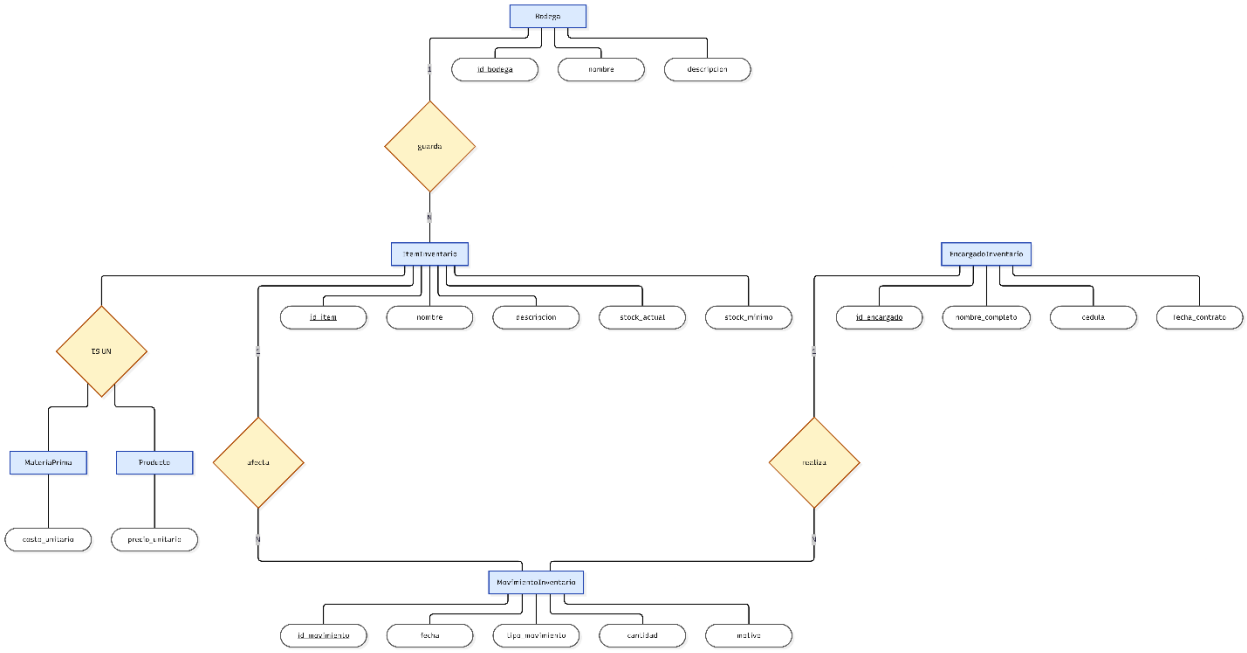


Ilustración 1. Diagrama Entidad-Relación del sistema de gestión de inventario de RainTamia Plus

Entidades y atributos (la llave primaria se puede observar en la tabla con la letra en negrita):

Entidad	Atributos
Bodega	**id_bodega**, nombre, descripcion
ItemInventario (superentidad)	**id_item**, nombre, descripcion, stock_actual, stock_minimo
MateriaPrima (subentidad)	costo_unitario
Producto (subentidad)	precio_unitario
EncargadoInventario	**id_encargado**, nombre_completo, cedula, fecha_contrato
MovimientoInventario	**id_movimiento**, fecha, tipo_movimiento, cantidad, motivo

Relaciones y cardinalidades:

Relación	Entidades	Cardinalidad
guarda	Bodega – ItemInventario	1:N
ES UN (especialización disjunta)	ItemInventario → MateriaPrima / Producto	1:1
afecta	ItemInventario – MovimientoInventario	1:N
realiza	EncargadoInventario – MovimientoInventario	1:N

4.2. Modelo Relacional

Tabla	Atributos (tipo sugerido)	PK	FK (referencia)
Bodega	id_bodega INT, nombre VARCHAR(80), descripcion VARCHAR(200)	id_bodega	—
ItemInventario	id_item INT, nombre VARCHAR(100), descripcion TEXT, stock_actual INT, stock_minimo INT, id_bodega INT	id_item	id_bodega → Bodega
MateriaPrima	id_item INT, costo_unitario DECIMAL(10,2)	id_item	id_item → ItemInventario
Producto	id_item INT, precio_unitario DECIMAL(10,2)	id_item	id_item → ItemInventario
EncargadoInventario	id_encargado INT, nombre_completo VARCHAR(120), cedula VARCHAR(10), fecha_contrato DATE	id_encargado	—
MovimientoInventario	id_movimiento INT, fecha TIMESTAMP, tipo_movimiento ENUM, cantidad INT, motivo TEXT, id_item INT, id_encargado INT	id_movimiento	id_item → ItemInventario; id_encargado → EncargadoInventario

Modelo generado con DBeaver

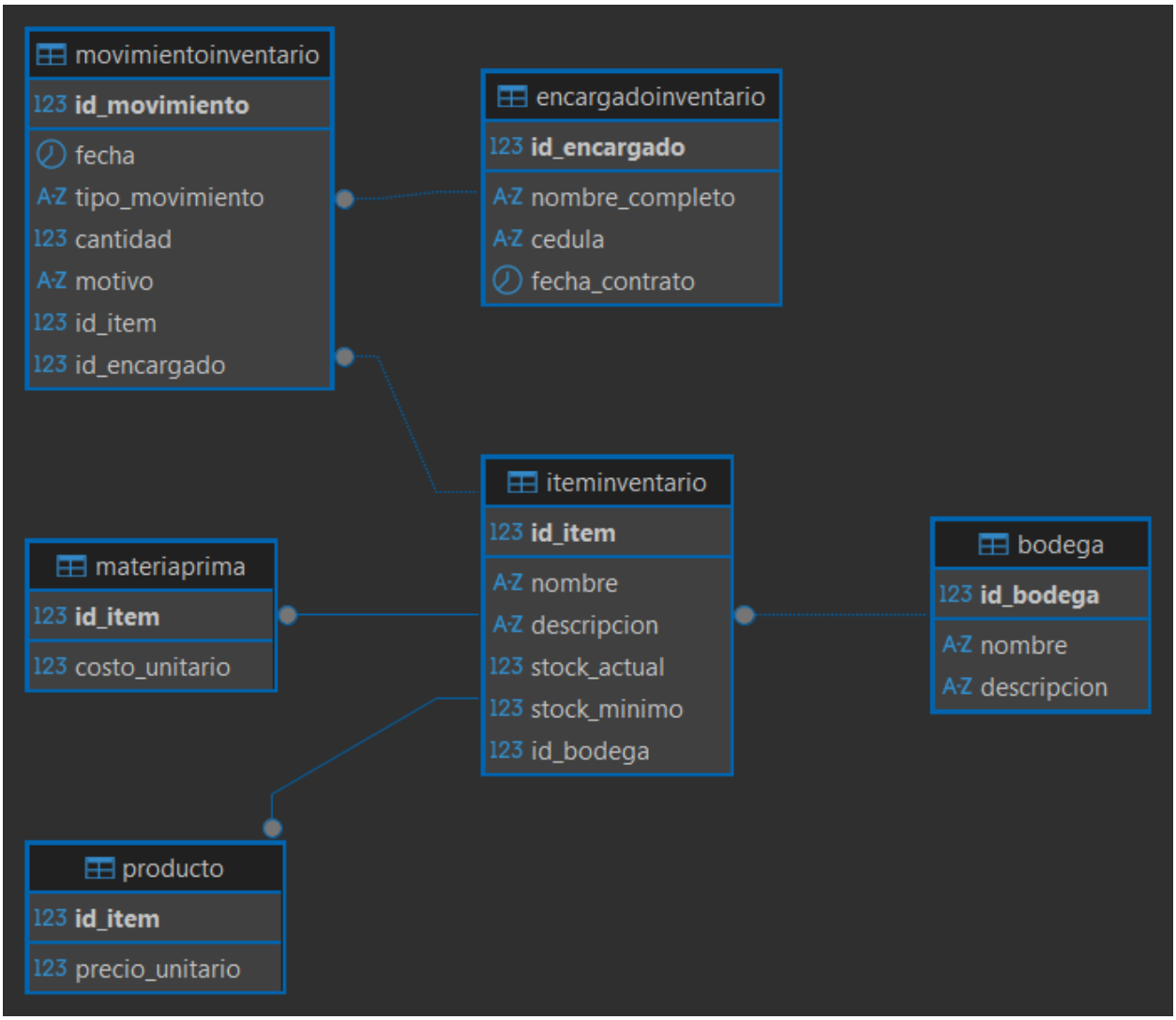


Ilustración 2. Esquema del modelo relacional generado en DBeaver.

CÓDIGO SQL

5.1. Creación de Tablas (DDL)

```
-- ============================================================
-- SCRIPT 01: DEFINICIÓN DE ESTRUCTURA DE DATOS (DDL)
-- Asignatura: Base de datos
-- Proyecto: Registro de inventario - RainTamia Plus
-- Motor de BD: PostgreSQL
```

```
-- Estudiante: Isaac David Ramón Tinoco
-- ============================================================

-- 1. Tipo enumerado para los tipos de movimiento de inventario
CREATE TYPE tipo_mov_enum AS ENUM ('ENTRADA', 'SALIDA', 'AJUSTE', 'PERDIDA');

-- 2. Tabla Bodega
CREATE TABLE Bodega (
   id_bodega   SERIAL,
   nombre      VARCHAR(80) NOT NULL,
   descripcion VARCHAR(200),
   PRIMARY KEY (id_bodega),
   CONSTRAINT uk_bodega_nombre UNIQUE (nombre)
);

-- 3. Tabla ItemInventario (Superentidad)
CREATE TABLE ItemInventario (
   id_item      SERIAL,
   nombre       VARCHAR(100) NOT NULL,
   descripcion  TEXT,
   stock_actual INT NOT NULL DEFAULT 0 CHECK (stock_actual >= 0),
   stock_minimo INT NOT NULL DEFAULT 0 CHECK (stock_minimo >= 0),
   id_bodega    INT NOT NULL,
   PRIMARY KEY (id_item),
   CONSTRAINT fk_item_bodega FOREIGN KEY (id_bodega) REFERENCES Bodega(id_bodega)
);

-- 4. Tabla MateriaPrima (Subentidad)
CREATE TABLE MateriaPrima (
   id_item       INT NOT NULL,
   costo_unitario DECIMAL(10,2) NOT NULL DEFAULT 0.00,
   PRIMARY KEY (id_item),
   CONSTRAINT chk_costo_positivo CHECK (costo_unitario >= 0),
   CONSTRAINT fk_materiaprima_item FOREIGN KEY (id_item) REFERENCES ItemInventario(id_item) ON DELETE
CASCADE
);

-- 5. Tabla Producto (Subentidad)
CREATE TABLE Producto (
   id_item        INT NOT NULL,
```

```
    precio_unitario DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    PRIMARY KEY (id_item),
    CONSTRAINT chk_precio_positivo CHECK (precio_unitario >= 0),
    CONSTRAINT fk_producto_item FOREIGN KEY (id_item) REFERENCES ItemInventario(id_item) ON DELETE
CASCADE
);

-- 6. Tabla EncargadoInventario
CREATE TABLE EncargadoInventario (
    id_encargado    SERIAL,
    nombre_completo VARCHAR(120) NOT NULL,
    cedula          VARCHAR(10) NOT NULL,
    fecha_contrato  DATE NOT NULL,
    PRIMARY KEY (id_encargado),
    CONSTRAINT uk_encargado_cedula UNIQUE (cedula)
);

-- 7. Tabla MovimientoInventario
CREATE TABLE MovimientoInventario (
    id_movimiento   SERIAL,
    fecha           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tipo_movimiento tipo_mov_enum NOT NULL,
    cantidad        INT NOT NULL,
    motivo          TEXT NOT NULL,
    id_item         INT NOT NULL,
    id_encargado    INT NOT NULL,
    PRIMARY KEY (id_movimiento),
    CONSTRAINT chk_cantidad_no_cero CHECK (cantidad <> 0),
    CONSTRAINT fk_movimiento_item FOREIGN KEY (id_item) REFERENCES ItemInventario(id_item),
    CONSTRAINT fk_movimiento_encargado FOREIGN KEY (id_encargado) REFERENCES
EncargadoInventario(id_encargado)
);
```

5.2. Inserción de Datos (DML)

Los datos son ficticios y coherentes entre sí: cinco bodegas, diez ítems (cinco materias primas y cinco productos), cinco encargados y diez movimientos. Todas las tablas tienen al menos cinco registros. Hay ítems bajo el mínimo, pérdidas, ajustes e ítems sin movimientos para que las consultas tengan resultados significativos.

```
-- ============================================================
```

```
-- SCRIPT 02: INSERCIÓN DE DATOS DE EJEMPLO (DML)
-- Asignatura: Base de datos
-- Proyecto: Registro de inventario - RainTamia Plus
-- Motor de BD: PostgreSQL
-- Estudiante: Isaac David Ramón Tinoco
-- ============================================================


-- 1. Inserción en Bodega
INSERT INTO Bodega (nombre, descripcion)
VALUES
   ('Bodega de envases y tapas', 'Almacena envases PET y tapas'),
   ('Bodega de producto terminado', 'Almacena botellas listas para la venta'),
   ('Bodega de insumos de empaque', 'Almacena sellos y etiquetas'),
   ('Área de despacho', 'Productos listos para entrega y botellones'),
   ('Área de producción', 'Insumos en uso en la línea de embotellado');

-- 2. Inserción en ItemInventario
INSERT INTO ItemInventario (id_item, nombre, descripcion, stock_actual, stock_minimo, id_bodega)
VALUES
   (1, 'Envase PET 600 ml', 'Envase plástico para agua de 600 ml', 450, 500, 1),
   (2, 'Tapa plástica 28 mm', 'Tapa rosca para envases PET', 3200, 1000, 1),
   (3, 'Sello de seguridad', 'Sello termoencogible para tapa', 900, 1000, 3),
   (4, 'Etiqueta adhesiva 600 ml', 'Etiqueta impresa RainTamia 600 ml', 5200, 1500, 3),
   (5, 'Funda termoencogible', 'Funda para armar pacas', 150, 100, 5),
   (6, 'Agua RainTamia 600 ml', 'Botella de agua de 600 ml', 1200, 400, 2),
   (7, 'Agua RainTamia 1 L', 'Botella de agua de 1 L', 300, 360, 2),
   (8, 'Agua RainTamia 2 L', 'Botella de agua de 2 L', 480, 200, 2),
   (9, 'Botellón RainTamia 20 L', 'Botellón retornable de 20 L', 60, 30, 4),
   (10, 'Agua RainTamia 330 ml', 'Botella de agua de 330 ml', 90, 120, 4);

-- Ajustar la secuencia del id_item por insertar llaves primarias de forma manual
SELECT setval('iteminventario_id_item_seq', (SELECT MAX(id_item) FROM ItemInventario));

-- 3. Inserción en MateriaPrima
INSERT INTO MateriaPrima (id_item, costo_unitario)
VALUES
   (1, 0.08),
   (2, 0.03),
   (3, 0.01),
   (4, 0.02),
```

```
    (5, 0.25);

-- 4. Inserción en Producto
INSERT INTO Producto (id_item, precio_unitario)
VALUES
    (6, 0.35),
    (7, 0.55),
    (8, 0.85),
    (9, 2.50),
    (10, 0.25);

-- 5. Inserción en EncargadoInventario
INSERT INTO EncargadoInventario (nombre_completo, cedula, fecha_contrato)
VALUES
    ('Marcos Villavicencio', '1104567891', '2022-03-01'),
    ('Lucía Armijos',        '1105678902', '2023-06-15'),
    ('Carlos Jaramillo',     '1106789013', '2021-09-10'),
    ('Andrea Ordóñez',       '1107890124', '2024-01-08'),
    ('Pablo Sarango',        '1108901235', '2025-02-20');

-- 6. Inserción en MovimientoInventario
INSERT INTO MovimientoInventario (fecha, tipo_movimiento, cantidad, motivo, id_item, id_encargado)
VALUES
    ('2026-10-05 08:15:00', 'ENTRADA', 5000, 'Compra de envases PET a proveedor', 1, 1),
    ('2026-10-05 08:40:00', 'ENTRADA', 4000, 'Compra de tapas plásticas', 2, 1),
    ('2026-10-12 14:20:00', 'SALIDA',  4550, 'Consumo en línea de producción', 1, 2),
    ('2026-10-14 09:05:00', 'PERDIDA', 100,  'Envases deformados por calor en bodega', 1, 2),
    ('2026-10-14 17:30:00', 'ENTRADA', 1500, 'Producción terminada lote L-1014', 6, 3),
    ('2026-10-16 10:10:00', 'SALIDA',  60,   'Venta a distribuidor', 6, 3),
    ('2026-10-19 11:45:00', 'PERDIDA', 30,   'Botellas rotas durante la distribución', 10, 4),
    ('2026-10-21 16:00:00', 'AJUSTE',  -5,   'Faltante en inventario físico', 9, 4),
    ('2026-10-23 09:30:00', 'AJUSTE',  200,  'Corrección de conteo de etiquetas', 4, 5),
    ('2026-10-26 13:15:00', 'SALIDA',  120,  'Venta a mayorista', 8, 5);
```

CONSULTA DE ÁLGEBRA RELACIONAL Y TRADUCIR A SQL

Consulta 1 — Selección, Proyección y Reunión (σ, π, ⋈)

Objetivo: identificar los ítems, junto con la bodega donde se encuentran, cuyo stock actual está por debajo del mínimo configurado, para decidir reabastecimientos (alertas de bajo stock).

Álgebra Relacional:

```
π ItemInventario.nombre, stock_actual, stock_minimo, Bodega.nombre (
    σ stock_actual < stock_minimo (
    ItemInventario ⋈ ItemInventario.id_bodega = Bodega.id_bodega Bodega
    )
)
```

SQL:

```
SELECT i.nombre AS item, i.stock_actual, i.stock_minimo, b.nombre AS bodega
FROM ItemInventario i
JOIN Bodega b ON i.id_bodega = b.id_bodega
WHERE i.stock_actual < i.stock_minimo;
```

Resultado esperado: cuatro ítems, cada uno con su bodega:

- Envase PET 600 ml (450 de 500, Bodega de envases y tapas),
- Sello de seguridad (900 de 1000, Bodega de insumos de empaque),
- Agua RainTamia 1 L (300 de 360, Bodega de producto terminado) y
- Agua RainTamia 330 ml (90 de 120, Área de despacho).

Resultado obtenido:

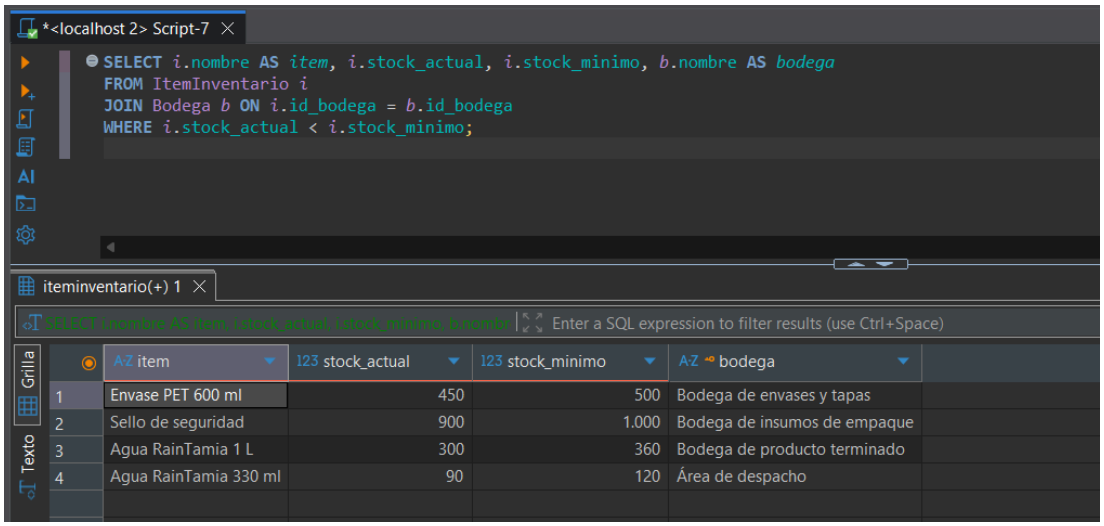


Ilustración 3. Resultado de la consulta de ítems con stock actual por debajo del mínimo.

Consulta 2 — Reunión natural de tres tablas (⋈) con selección

Objetivo: listar las salidas y pérdidas registradas, indicando qué ítem se afectó y qué encargado las registró (trazabilidad de responsables).

Álgebra Relacional:

```
π nombre, tipo_movimiento, cantidad, nombre_completo, fecha (
    σ tipo_movimiento = 'SALIDA' ∨ tipo_movimiento = 'PERDIDA' (
    (ItemInventario ⋈ ItemInventario.id_item = MovimientoInventario.id_item MovimientoInventario)
```

```
    ⋈ MovimientoInventario.id_encargado = EncargadoInventario.id_encargado EncargadoInventario
    )
)
```

SQL:

```
SELECT i.nombre, m.tipo_movimiento, m.cantidad, e.nombre_completo, m.fecha
FROM ItemInventario i
JOIN MovimientoInventario m ON i.id_item = m.id_item
JOIN EncargadoInventario e ON m.id_encargado = e.id_encargado
WHERE m.tipo_movimiento IN ('SALIDA', 'PERDIDA')
ORDER BY m.fecha;
```

Resultado esperado: cinco filas: tres salidas (Envase PET 600 ml, Agua 600 ml y Agua 2 L) y dos pérdidas (Envase PET 600 ml y Agua 330 ml), cada una con el nombre del encargado y la fecha.

Resultado obtenido:

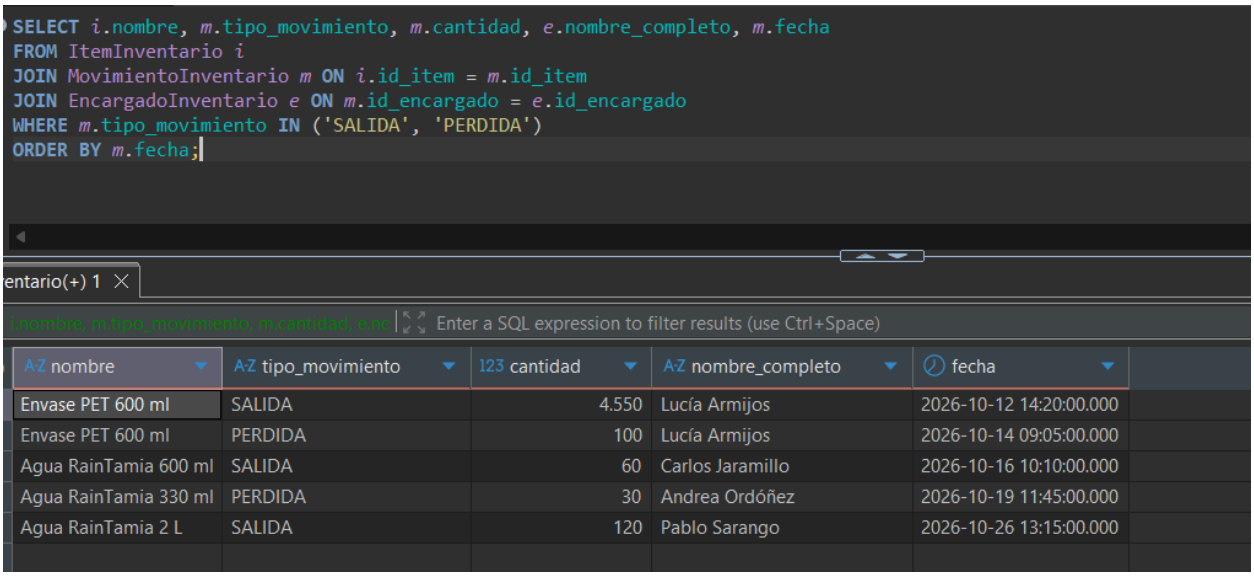


Ilustración 4. Resultado de la consulta de trazabilidad de salidas y pérdidas por responsable.

Consulta 3 — Diferencia (−)

Objetivo: encontrar las materias primas que no han tenido ningún movimiento registrado, es decir, inventario sin rotación o sin control.

Álgebra Relacional:

```
R = π id_item (MateriaPrima) − π id_item (MovimientoInventario)
π nombre (ItemInventario ⋈ R)
```

SQL:

```
SELECT i.nombre
FROM ItemInventario i
JOIN MateriaPrima m ON i.id_item = m.id_item
WHERE NOT EXISTS (
    SELECT 1
    FROM MovimientoInventario mv
    WHERE mv.id_item = m.id_item
);
```

Resultado esperado:

- Sello de seguridad
- Funda termoencogible

Resultado obtenido:

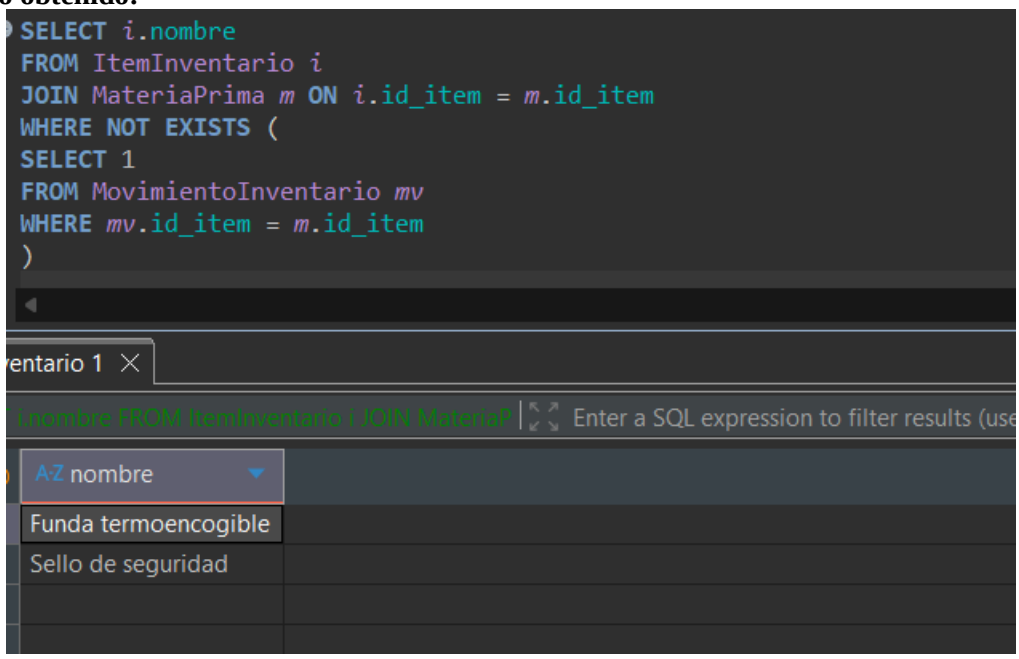


Ilustración 5. Resultado de la consulta de materias primas sin movimientos registrados.

Consulta 4 — Intersección (∩)

Objetivo: obtener los productos que tuvieron al menos una entrada (producción) y también al menos una salida (venta), para identificar los productos que realmente rotan.

Álgebra Relacional:

```
A = π id_item (σ tipo_movimiento = 'ENTRADA' (MovimientoInventario))
B = π id_item (σ tipo_movimiento = 'SALIDA' (MovimientoInventario))
π nombre (ItemInventario ⋈ Producto ⋈ (A ∩ B))
```

SQL:

```
SELECT i.nombre
FROM ItemInventario i
JOIN Producto p ON i.id_item = p.id_item
WHERE i.id_item IN (SELECT id_item FROM MovimientoInventario WHERE tipo_movimiento = 'ENTRADA')
    AND i.id_item IN (SELECT id_item FROM MovimientoInventario WHERE tipo_movimiento = 'SALIDA');
```

Resultado esperado: Agua RainTamia 600 ml, que tuvo una entrada de 1500 unidades (producción) y una salida de 60 (venta).

Resultado obtenido:

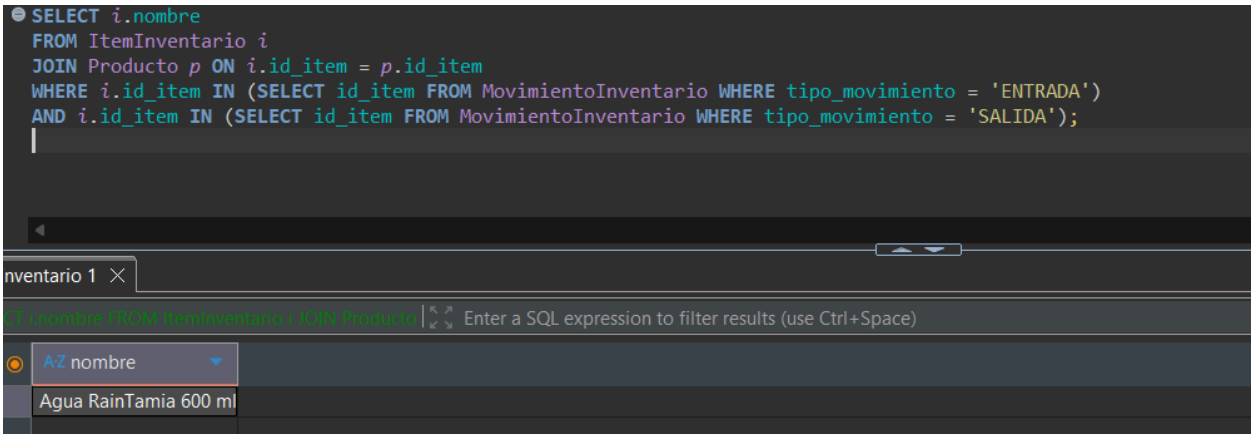


Ilustración 6. Resultado de la consulta de productos con entradas y salidas (rotación).

Consulta 5 — Agrupamiento y funciones de agregación (γ)

Objetivo: calcular el valor monetario del stock de materia prima por bodega (stock actual por costo unitario) y mostrar solo las bodegas que superan los 100 dólares, para apoyar la valoración y el control de costos del inventario.

Álgebra Relacional:

```
σ valor_total > 100 (
    γ Bodega.nombre; SUM(stock_actual × costo_unitario) → valor_total (
    (Bodega ⋈ Bodega.id_bodega = ItemInventario.id_bodega ItemInventario)
    ⋈ ItemInventario.id_item = MateriaPrima.id_item MateriaPrima
    )
)
```

SQL:

```
SELECT b.nombre AS bodega,
    SUM(i.stock_actual * m.costo_unitario) AS valor_total
FROM Bodega b
JOIN ItemInventario i ON b.id_bodega = i.id_bodega
JOIN MateriaPrima m ON i.id_item = m.id_item
```

```
GROUP BY b.id_bodega, b.nombre
HAVING SUM(i.stock_actual * m.costo_unitario) > 100
ORDER BY valor_total DESC;
```

Resultado esperado: Bodega de envases y tapas con 132,00 dólares y Bodega de insumos de empaque con 113,00. El Área de producción (37,50) queda fuera por no superar los 100.

Resultado obtenido:

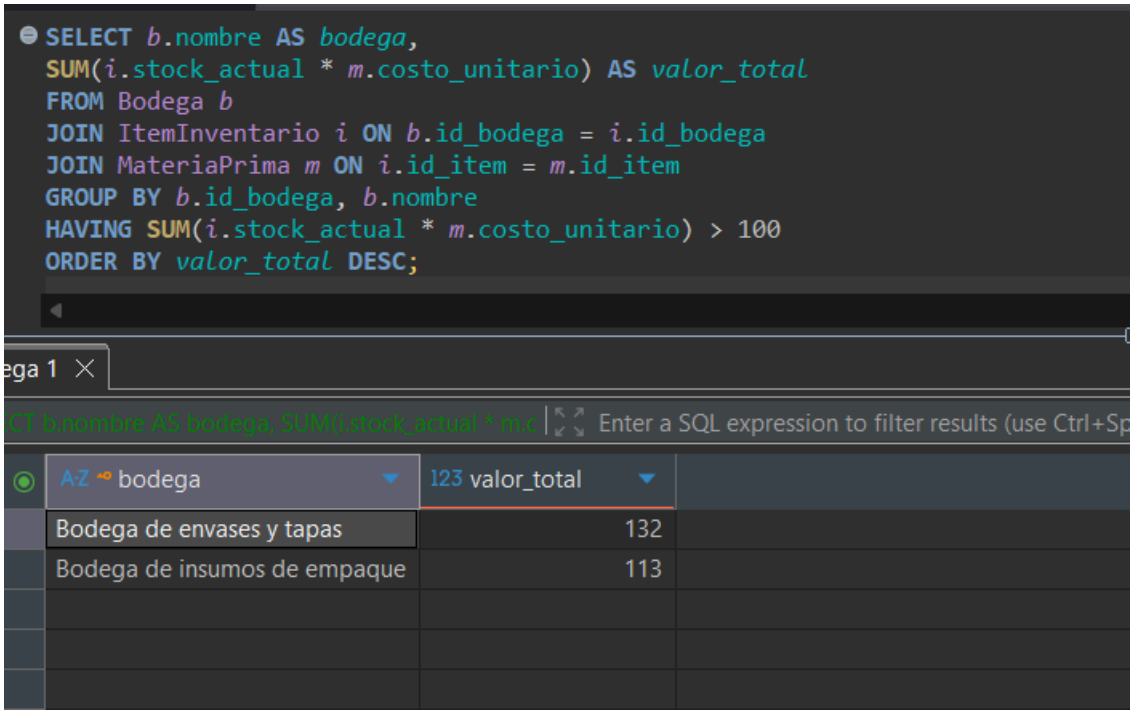


Ilustración 7. Resultado de la consulta del valor monetario del inventario de materia prima por bodega.

CONCLUSIONES

- Modelar el registro de inventario con seis tablas (Bodega, ItemInventario, MateriaPrima, Producto, EncargadoInventario y MovimientoInventario) permitió separar lo común de los ítems (nombre y stock) de lo propio de cada tipo (el costo en la materia prima y el precio en el producto) mediante una especialización. Esta decisión evita campos vacíos y deja que cada movimiento apunte a cualquier ítem con una sola llave foránea. La mayor dificultad fue identificar la relación entre movimientos, ítems y responsables, porque la trazabilidad de las pérdidas depende de saber qué se movió, quién lo registró y por qué.
- Escribir el DDL y el DML permitió comprobar que el modelo funcionaba: las llaves foráneas obligaron a insertar los datos en el orden correcto y las restricciones CHECK, UNIQUE y los tipos ENUM impiden registrar tipos de movimiento o cantidades inválidas. Esto se relaciona directamente con la transparencia y la responsabilidad, porque cada movimiento queda asociado a un ítem, un encargado, una fecha y un motivo. Lo más relevante del lenguaje fue la definición de la integridad referencial y de restricciones que protegen la calidad de los datos desde la base.

- Formular primero cada consulta en álgebra relacional ayudó a razonar qué operación se necesitaba antes de escribir SQL: selección para el stock bajo, reunión para trazar responsables, diferencia e intersección para analizar qué ítems tienen o no movimientos, y agrupamiento para resumir. Las consultas más útiles para el inventario fueron las de stock bajo y las que relacionan movimientos con ítems y encargados, porque respaldan decisiones de reabastecimiento y permiten rastrear pérdidas, que eran los problemas planteados en el contexto.

BIBLIOGRAFÍA

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.
[3] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: https://www.postgresql.org/docs/. [Accessed: Nov. 2026].